

LAPORAN TUGAS AKHIR UAS

Perancangan dan Implementasi Sistem Robotika Berbasis Intelligent Systems

[LOGO KAMPUS]

(Simpan file logo.png di
folder figures/ lalu hapus
box ini)

Disusun Oleh (Kelompok UAS):

Nama		NIM
Nama Anggota 1	—	1234567890
Nama Anggota 2	—	1234567891
Nama Anggota 3	—	1234567892
Nama Anggota 4	—	1234567893

Dosen Pengampu: Nama Dosen Pengampu, M.T.

Intelligent Systems and Robotics (ISR)

Teknik Elektro / Teknik Komputer / Informatika

Fakultas Teknik / Ilmu Komputer

Universitas Nama Kampus

Tahun 2025/2026

Daftar Isi

1	Pendahuluan	1
1.1	Latar Belakang	1
1.2	Rumusan Masalah	1
1.3	Tujuan Penelitian	1
1.4	Manfaat Penelitian	1
2	Tinjauan Pustaka	3
2.1	Sistem Kendali Robot (Intelligent Controls)	3
2.2	Sensor Jarak dan Penginderaan Lingkungan	3
2.3	Platform Mikrokontroler / Single Board Computer	3
3	Perancangan Sistem	4
3.1	Perancangan Perangkat Keras (Hardware)	4
3.2	Perancangan Sistem Visual (Visual)	5
3.3	Perancangan Logika Kontrol dan Navigasi (Logic)	6
3.3.1	Pengendali Driver Motor (motor_manager.py)	6
3.3.2	Pengelola Sensor Jarak Ultrasonik (ultrasonic_manager.py)	8
3.3.3	Algoritma Kontrol PID dan Logika Navigasi Utama (navigator.py)	11
4	Pengujian dan Hasil Pembahasan	19
4.1	Pengujian Akurasi Sensor Jarak	19
4.2	Analisis Pengujian Navigasi	19
5	Kesimpulan dan Saran	20
5.1	Kesimpulan	20
5.2	Saran	20

Daftar Gambar

1	Diagram Blok Arsitektur Perangkat Keras Robot.	4
2	Dokumentasi Fisik Robot Hasil Perakitan Kelompok.	5

Daftar Tabel

1	Data Pengujian Akurasi Sensor Jarak Ultrasonik HC-SR04.	19
---	---	----

1 Pendahuluan

1.1 Latar Belakang

Tuliskan latar belakang topik proyek robotika atau sistem cerdas yang Anda kerjakan di sini. Latar belakang harus mencakup penjelasan mengenai permasalahan riil yang melandasi pentingnya pembuatan sistem robot ini, penelitian terdahulu yang relevan, serta solusi cerdas yang diajukan dalam proyek ini.

Contoh pengacuan pustaka menggunakan LaTeX: Robot ini dirancang untuk mengatasi permasalahan navigasi otonom menggunakan algoritma tertentu [1].

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian/proyek ini adalah:

1. Bagaimana merancang arsitektur perangkat keras dan lunak pada robot agar mampu beroperasi secara cerdas?
2. Bagaimana performa algoritma kontrol/navigasi yang diimplementasikan pada sistem robot tersebut?
3. Sejauh mana tingkat akurasi pembacaan sensor dalam mengenali kondisi lingkungan sekitar?

1.3 Tujuan Penelitian

Tujuan dari pelaksanaan proyek UAS Intelligent Systems and Robotics ini adalah:

1. Merancang dan mengimplementasikan sistem robotika terintegrasi yang mampu melakukan [tuliskan tugas spesifik robot].
2. Menganalisis kinerja sensor dan aktuator yang digunakan pada platform robot.
3. Menguji keandalan sistem kontrol cerdas dalam skenario lingkungan nyata.

1.4 Manfaat Penelitian

Hasil dari rancang bangun sistem robotika ini diharapkan dapat memberikan manfaat sebagai berikut:

- **Bagi Mahasiswa:** Meningkatkan pemahaman praktis mengenai integrasi sensor, aktuator, dan algoritma kecerdasan buatan pada sistem fisik.
- **Bagi Akademisi:** Menjadi referensi tambahan dalam pengembangan sistem navigasi robotika otonom skala laboratorium.
- **Bagi Praktisi:** Memberikan inspirasi desain sistem robotika berbiaya rendah dengan performa yang andal.

2 Tinjauan Pustaka

Bagian ini memaparkan teori dasar yang mendukung pengerjaan proyek robotika ini. Teori-teori ini mencakup sensor yang digunakan, sistem kendali, serta algoritma kecerdasan buatan yang ditanamkan pada robot.

2.1 Sistem Kendali Robot (Intelligent Controls)

Sistem kontrol pada robot sering menggunakan algoritma seperti Proportional-Integral-Derivative (PID), Fuzzy Logic, atau kecerdasan buatan lainnya. Kontrol cerdas ini memungkinkan robot untuk mengambil keputusan secara dinamis berdasarkan data sensor secara real-time.

2.2 Sensor Jarak dan Penginderaan Lingkungan

Untuk menavigasi lingkungan, robot dilengkapi dengan sensor jarak seperti Ultrasonik (HC-SR04), LiDAR, atau kamera. Persamaan dasar perhitungan jarak pada sensor ultrasonik adalah:

$$s = \frac{v \times t}{2} \quad (1)$$

di mana s adalah jarak objek (meter), v adalah kecepatan rambat suara di udara (≈ 340 m/s), dan t adalah waktu tempuh gelombang pantul (detik).

2.3 Platform Mikrokontroler / Single Board Computer

Robot ini menggunakan platform mikrokontroler [tuliskan nama board, misal: Arduino Uno, ESP32, Raspberry Pi, Jetson Nano] sebagai unit pemroses utama. Unit ini bertugas membaca data input dari sensor, melakukan pemrosesan algoritma navigasi, dan memberikan sinyal kendali PWM (Pulse Width Modulation) ke driver motor.

3 Perancangan Sistem

Pada bab ini dijelaskan arsitektur perancangan sistem robotika secara menyeluruh, yang terbagi menjadi perancangan perangkat keras (hardware), perancangan sistem visual (visi komputer), dan perancangan logika kontrol navigasi (logic).

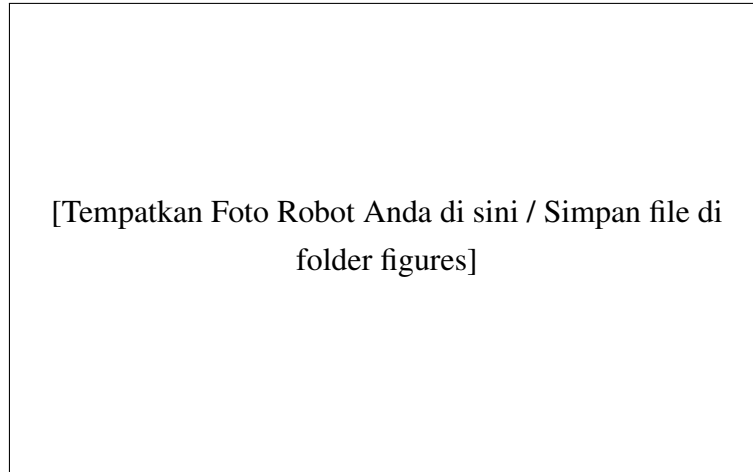
3.1 Perancangan Perangkat Keras (Hardware)

Robot ini dirancang menggunakan platform *Single Board Computer* (SBC) berupa Raspberry Pi sebagai unit pemroses utama. Perangkat keras robot terbagi menjadi komponen mekanik sasis, sistem penggerak aktuator berupa motor DC dengan driver motor L298N, sensor jarak ultrasonik HC-SR04 untuk deteksi rintangan fisik jarak dekat, serta USB Webcam sebagai sensor penglihatan utama. Diagram blok sistem perangkat keras dapat dilihat pada Gambar 1.



Gambar 1: Diagram Blok Arsitektur Perangkat Keras Robot.

Dokumentasi foto robot yang telah dirakit diletakkan pada Gambar 2. File gambar disimpan di dalam folder `figures/`.



Gambar 2: Dokumentasi Fisik Robot Hasil Perakitan Kelompok.

3.2 Perancangan Sistem Visual (Visual)

Perancangan subsistem visual berfokus pada pemrosesan citra digital yang ditangkap oleh USB Webcam secara real-time. Webcam diposisikan di bagian depan robot untuk menangkap kondisi lingkungan. Sinyal video dari webcam diproses oleh modul pemrosesan citra (Visi Komputer) yang dikembangkan terpisah dan dijalankan sebagai layanan web server lokal berbasis Flask API pada alamat URL `http://127.0.0.1:5000/api/objects`.

Layanan Flask API ini mengekstraksi frame gambar, melakukan segmentasi atau deteksi objek, dan membagi objek yang terdeteksi ke dalam dua kategori label utama, yaitu:

1. **TARGET:** Representasi objek tujuan yang harus dikejar oleh robot (misalnya bola berwarna atau penanda jalur khusus).
2. **OBSTACLE:** Representasi rintangan visual di depan robot yang harus dihindari secara visual sebelum terlalu dekat secara fisik.

Data hasil pemrosesan citra dikembalikan dalam format JSON yang berisi koordinat bounding box setiap objek. Parameter penting yang digunakan oleh sistem navigasi adalah koordinat tengah sumbu-x objek (x_{objek}) untuk menentukan deviasi arah, serta ukuran dimensi lebar (w) dan tinggi (h) bounding box untuk menghitung luas area objek ($A = w \times h$) sebagai indikator jarak visual objek dari kamera.

3.3 Perancangan Logika Kontrol dan Navigasi (Logic)

Bagian perancangan logika kontrol merupakan pusat pengambilan keputusan robot berbasis Raspberry Pi. Logika navigasi dirancang menggunakan bahasa pemrograman Python 3 dengan struktur modular yang terdiri dari tiga berkas program utama: `motor_manager.py`, `ultrasonic_manager.py`, dan `navigator.py`. Diagram alir atau arsitektur modular perangkat lunak navigasi ini diintegrasikan dalam satu kesatuan sistem otonom.

3.3.1 Pengendali Driver Motor (`motor_manager.py`)

Modul `motor_manager.py` bertanggung jawab langsung untuk mengontrol arah putaran dan kecepatan motor DC menggunakan sinyal PWM (*Pulse Width Modulation*) yang disuplai oleh Raspberry Pi melalui GPIO ke driver motor L298N. Kode implementasi modul ini dapat dilihat pada Kode Program 1.

Beberapa aspek logika penting pada modul ini meliputi:

- **Kalibrasi Bias Roda (*Left Bias*):** Karena ketidakseimbangan mekanis bawaan motor DC kiri dan kanan, diterapkan pengali bias roda kiri (`left_bias`) sebesar 1.14 pada saat inisialisasi di `navigator` agar robot dapat melaju lurus secara simetris.
- **Pembatasan Kecepatan Proporsional (*Speed Limiting*):** Kecepatan dikontrol dalam rentang -255 hingga 255 . Jika input kecepatan melebihi batas `max_speed_limit` (disetel pada 180), kecepatan kedua motor diturunkan secara proporsional menggunakan persamaan rasio agar arah belokan (*steering ratio*) tetap terjaga sesuai instruksi navigasi.
- **Torsi Minimum Motor:** Untuk mengatasi gesekan statis sasis robot, diterapkan logika ambang batas bawah di mana motor yang diinstruksikan bergerak akan diberikan daya minimal sebesar ± 90 .

```
1 import RPi.GPIO as GPIO
2
3 class MotorManager:
4     def __init__(self, ena=13, in1=19, in2=26, enb=12, in3=16, in4=20,
5         max_speed_limit=150, left_bias=1.0):
6         # BCM Pin Configuration
7         self.ENA = ena # PWM Left
8         self.IN1 = in1 # Dir 1 Left
9         self.IN2 = in2 # Dir 2 Left
10        self.ENB = enb # PWM Right
```

```

11     self.IN3 = in3 # Dir 1 Right
12     self.IN4 = in4 # Dir 2 Right
13
14     self.max_speed_limit = max_speed_limit
15     self.left_bias = left_bias
16
17     # Setup Pins
18     GPIO.setmode(GPIO.BCM)
19     for pin in [self.ENA, self.IN1, self.IN2, self.ENB, self.IN3,
20 self.IN4]:
21         GPIO.setup(pin, GPIO.OUT)
22
23     # Setup PWM
24     self.pwm_left = GPIO.PWM(self.ENA, 1000) # 1kHz frequency
25     self.pwm_right = GPIO.PWM(self.ENB, 1000)
26
27     self.pwm_left.start(0)
28     self.pwm_right.start(0)
29
30 def set_motors(self, speed_left, speed_right):
31     """
32     Set speed for both motors.
33     speed range: -255 to 255 (negative for reverse)
34     """
35     # Terapkan bias untuk motor kiri
36     speed_left = speed_left * self.left_bias
37
38     # Batasi kecepatan secara proporsional agar rasio kemudi tetap
39     terjaga
40     # Gunakan max_speed_limit sebagai plafon absolut
41     abs_l = abs(speed_left)
42     abs_r = abs(speed_right)
43     max_val = max(abs_l, abs_r)
44
45     if max_val > self.max_speed_limit:
46         ratio = self.max_speed_limit / max_val
47         speed_left *= ratio
48         speed_right *= ratio
49
50     # --- Left Motor ---
51     if speed_left >= 0:
52         GPIO.output(self.IN1, GPIO.HIGH)
53         GPIO.output(self.IN2, GPIO.LOW)
54     else:
55         GPIO.output(self.IN1, GPIO.LOW)
56         GPIO.output(self.IN2, GPIO.HIGH)

```

```

54         speed_left = abs(speed_left)
55
56         # Convert 0-255 to 0-100% duty cycle
57         duty_left = (speed_left / 255.0) * 100
58         self.pwm_left.ChangeDutyCycle(min(duty_left, 100))
59
60         # --- Right Motor ---
61         if speed_right >= 0:
62             GPIO.output(self.IN3, GPIO.HIGH)
63             GPIO.output(self.IN4, GPIO.LOW)
64         else:
65             GPIO.output(self.IN3, GPIO.LOW)
66             GPIO.output(self.IN4, GPIO.HIGH)
67             speed_right = abs(speed_right)
68
69         duty_right = (speed_right / 255.0) * 100
70         self.pwm_right.ChangeDutyCycle(min(duty_right, 100))
71
72     def stop(self):
73         self.pwm_left.ChangeDutyCycle(0)
74         self.pwm_right.ChangeDutyCycle(0)
75         GPIO.output(self.IN1, GPIO.LOW)
76         GPIO.output(self.IN2, GPIO.LOW)
77         GPIO.output(self.IN3, GPIO.LOW)
78         GPIO.output(self.IN4, GPIO.LOW)
79
80     def cleanup(self):
81         self.stop()
82         self.pwm_left.stop()
83         self.pwm_right.stop()
84         self.pwm_left = None
85         self.pwm_right = None

```

Listing 1: Kode Program Modul Pengendali Motor (motor_manager.py)

3.3.2 Pengelola Sensor Jarak Ultrasonik (ultrasonic_manager.py)

Modul `ultrasonic_manager.py` digunakan untuk berinteraksi dengan sensor ultrasonik HC-SR04 guna membaca jarak fisik hambatan di depan robot secara langsung. Kode implementasi modul ini ditunjukkan pada Kode Program 2.

Untuk memastikan akurasi dan kecepatan respon navigasi, modul ini menerapkan beberapa metode logis berikut:

- **Moving Average dan Penyaringan Data (Filtering):** Sensor melakukan 3 kali pembacaan jarak mentah dengan jeda singkat 10 ms antar pembacaan. Nilai yang dianggap valid adalah jarak dalam rentang 2 hingga 400 cm. Pembacaan yang valid kemudian dirata-ratakan untuk menyaring pencilaan (*noise*).
- **Faktor Koreksi Dinamis (Kalibrasi):** Modul memuat nilai faktor koreksi dari berkas teks eksternal `faktor_koreksi.txt` saat inisialisasi. Jarak akhir yang digunakan oleh robot merupakan hasil perkalian jarak mentah rata-rata dengan faktor koreksi ini ($s_{\text{kalibrasi}} = s_{\text{mentah}} \times f_{\text{koreksi}}$).
- **Pencegahan Kegagalan (Fallback):** Jika sensor gagal mendapatkan data pembacaan yang valid, fungsi secara otomatis mengembalikan nilai 999 cm agar robot tidak berhenti mendadak akibat kesalahan sensor sementara.

```

1 import RPi.GPIO as GPIO
2 import time
3 import os
4
5 class UltrasonicManager:
6     def __init__(self, trigger_pin=17, echo_pin=27, file_kalibrasi="
    faktor_koreksi.txt"):
7         self.PIN_TRIGGER = trigger_pin
8         self.PIN_ECHO = echo_pin
9         self.FILE_KALIBRASI = file_kalibrasi
10        self.faktor_koreksi = 1.0
11
12        # Setup GPIO
13        GPIO.setmode(GPIO.BCM)
14        GPIO.setup(self.PIN_TRIGGER, GPIO.OUT)
15        GPIO.setup(self.PIN_ECHO, GPIO.IN)
16        GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
17
18        self.load_kalibrasi()
19
20    def load_kalibrasi(self):
21        if os.path.exists(self.FILE_KALIBRASI):
22            try:
23                with open(self.FILE_KALIBRASI, "r") as f:
24                    self.faktor_koreksi = float(f.read().strip())
25                    print(f"[ULTRASONIC] Faktor koreksi dimuat: {self.
    faktor_koreksi:.4f}")
26            except:
27                print("[ULTRASONIC] Gagal membaca file kalibrasi,
    menggunakan 1.0")

```

```

28
29 def ukur_jarak_mentah(self):
30     total_jarak = 0
31     pembacaan_valid = 0
32
33     for _ in range(3): # Dikurangi ke 3 pembacaan agar navigasi
lebih responsif
34         GPIO.output(self.PIN_TRIGGER, GPIO.HIGH)
35         time.sleep(0.00001)
36         GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
37
38         waktu_mulai = time.time()
39         waktu_selesai = time.time()
40         timeout = time.time() + 0.1 # Timeout dipercepat
41
42         while GPIO.input(self.PIN_ECHO) == 0:
43             waktu_mulai = time.time()
44             if waktu_mulai > timeout: break
45
46         while GPIO.input(self.PIN_ECHO) == 1:
47             waktu_selesai = time.time()
48             if waktu_selesai > timeout: break
49
50         durasi = waktu_selesai - waktu_mulai
51         jarak = (durasi * 34300) / 2
52
53         if 2 <= jarak <= 400:
54             total_jarak += jarak
55             pembacaan_valid += 1
56             time.sleep(0.01)
57
58         if pembacaan_valid > 0:
59             return total_jarak / pembacaan_valid
60         return None
61
62 def get_jarak(self):
63     mentah = self.ukur_jarak_mentah()
64     if mentah is not None:
65         return mentah * self.faktor_koreksi
66     return 999 # Kembalikan nilai jauh jika gagal baca agar robot
tidak stop tiba-tiba
67
68 def cleanup(self):
69     pass

```

Listing 2: Kode Program Modul Sensor Ultrasonik (ultrasonic_manager.py)

3.3.3 Algoritma Kontrol PID dan Logika Navigasi Utama (navigator.py)

Modul `navigator.py` merupakan otak navigasi otonom yang mengintegrasikan data sensor jarak ultrasonik dan data visi komputer untuk mengambil keputusan gerak secara sekuensial. Kode lengkap dari modul ini ditunjukkan pada Kode Program 3.

Algoritma Kendali PID (Proportional-Integral-Derivative) Sistem kemudi otomatis saat mendeteksi target visual menggunakan pengendali PID untuk meminimalkan deviasi arah (error) terhadap titik tengah kamera. Persamaan matematika PID yang diimplementasikan adalah:

$$e(t) = x_{\text{target}} - x_{\text{setpoint}} \quad (2)$$

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3)$$

di mana $x_{\text{setpoint}} = 160$ piksel (nilai tengah horizontal dari resolusi lebar citra 320 piksel), dan $e(t)$ adalah error posisi horizontal target. Nilai konstanta kendali yang disetel adalah $K_p = 0.8$ untuk respon proporsional yang tanggap, $K_d = 0.05$ untuk meredam osilasi arah, dan $K_i = 0.0$ untuk menghindari akumulasi error jangka panjang. Luaran kendali $u(t)$ bertindak sebagai nilai kemudi (*steer value*) yang memodifikasi kecepatan motor kiri (v_{kiri}) dan motor kanan (v_{kanan}):

$$v_{\text{kiri}} = V_{\text{base}} + u(t) \quad (4)$$

$$v_{\text{kanan}} = V_{\text{base}} - u(t) \quad (5)$$

di mana $V_{\text{base}} = 125$ merupakan kecepatan dasar gerakan maju robot.

Arsitektur Finite State Machine (FSM) Sekuensial Logika navigasi beroperasi dalam suatu loop utama tak terbatas yang berjalan secara sekuensial dan terbagi menjadi beberapa keadaan (*states*):

1. **State 1: Deteksi Rintangan Ultrasonik Jarak Dekat:** Jarak depan diperiksa terlebih dahulu menggunakan sensor ultrasonik.
 - Jika $s \leq 15$ cm, robot akan berhenti seketika.
 - Sistem memeriksa data visi komputer. Jika objek di depannya terdeteksi sebagai TARGET dengan area visual > 10.000 piksel dan jarak fisik ≤ 12 cm, robot mendeteksi kondisi **Finish** dan menghentikan navigasi sepenuhnya.

- Jika bukan target akhir, robot mendeteksi rintangan (OBSTACLE) fisik. Robot mundur perlahan ($-V_{SLOW}$) selama 0.5 detik, berhenti 0.2 detik, kemudian melakukan manuver berputar menghindari ke kiri atau kanan selama 0.3 detik berdasarkan arah memori target terakhir (`last_known_target_direction`).

2. **State 2: Pembacaan Target dan Rintangan Visual:** Jika tidak ada rintangan jarak dekat, robot berhenti diam selama 0.1 detik untuk menstabilkan gambar kamera dari efek kabur akibat guncangan (*motion blur*), memanggil REST API untuk mengambil data visi terbaru, dan mengidentifikasi target terbesar dengan area > 600 piksel serta rintangan visual terbesar dengan area > 2000 piksel.

3. State 3: Eksekusi Gerakan Berprioritas:

- **Prioritas 1 (Pengejaran Target):** Jika target terdeteksi, robot melaju ke arah target menggunakan kendali kemudi PID yang dihitung dari koordinat target. Arah gerak disimpan ke variabel memori `last_known_target_direction` (LEFT atau RIGHT). Robot melaju selama 0.25 detik.
- **Prioritas 2 (Penghindaran Rintangan Visual):** Jika tidak ada target tetapi rintangan terdeteksi di area tengah (160 ± 75 piksel), robot menghindari dengan berbelok menjauh dari rintangan visual tersebut. Jika rintangan berada di luar area tengah, robot melaju lurus dengan aman. Robot melaju selama 0.25 detik.
- **Prioritas 3 (Pencarian Target berbasis Memori):** Jika tidak ada objek terdeteksi, robot memutar badannya ke arah memori target terakhir untuk menemukannya kembali selama 0.2 detik. Jika tidak ada memori arah target, robot diam demi keamanan.

```

1 import requests
2 import time
3 from .motor_manager import MotorManager
4 from .ultrasonic_manager import UltrasonicManager
5
6 #
7 # =====
8 #
9 #
10 # --- Parameter PID (Digunakan untuk kalkulasi steer) ---
11 Kp = 0.8 # Lebih agresif

```

```

12 Ki = 0.0
13 Kd = 0.05
14
15 # --- Pengaturan Kecepatan Motor (PWM 0-255) ---
16 V_BASE = 125    # Diturunkan sedikit agar selisih kecepatan lebih terasa
17 V_SLOW = 100
18 MAX_SPEED_LIMIT = 180
19
20 # --- Konstanta Kamera ---
21 SETPOINT_TENGAH = 160
22
23 # --- Koneksi Visi ---
24 API_URL = "http://127.0.0.1:5000/api/objects"
25
26 #
27 # =====
28 #
29 #
30 # =====
31
32 last_error = 0
33 integral = 0
34
35 def hitung_PID(current_error):
36     global last_error, integral
37     P = current_error
38     integral += current_error
39     if integral > 50: integral = 50
40     if integral < -50: integral = -50
41     derivative = current_error - last_error
42     total_output = (Kp * P) + (Ki * integral) + (Kd * derivative)
43     last_error = current_error
44     return total_output
45
46 # --- Init Hardware ---
47 # Bias 1.14 untuk mengimbangi roda kiri secara permanen
48 motors = MotorManager(max_speed_limit=MAX_SPEED_LIMIT, left_bias=1.14)
49 ultrasonic = UltrasonicManager()
50
51 def jalankan_motor(v_kiri, v_kanan):
52     # Pastikan motor yang harusnya jalan punya torsi cukup (min 90)
53     if v_kiri > 0 and v_kiri < 90: v_kiri = 90
54     if v_kanan > 0 and v_kanan < 90: v_kanan = 90
55     if v_kiri < 0 and v_kiri > -90: v_kiri = -90

```



```

53     if v_kanan < 0 and v_kanan > -90: v_kanan = -90
54
55     motors.set_motors(v_kiri, v_kanan)
56
57 def stop_robot():
58     motors.stop()
59
60 def ambil_data_visi():
61     try:
62         r = requests.get(API_URL, timeout=0.1)
63         if r.status_code == 200:
64             return r.json()
65     except:
66         pass
67     return None
68
69 #
=====
70 # LOOP UTAMA (LOGIKA SEKUENSIAL: AMBIL GAMBAR -> JALAN -> STOP)
71 #
=====
72
73 def main_loop(shared_data=None):
74     SAFE_MARGIN_PIXELS = 100
75     MIN_AREA_TARGET = 600          # Abaikan target terlalu kecil/jauh
76     MIN_AREA_OBSTACLE = 2000       # Abaikan rintangan jauh
77     TARGET_REACHED_AREA = 10000    # Target dianggap "Penuh" di kamera (
Finish)
78
79     print(f"Memulai Navigasi Sekuensial (Smooth | Bias: 1.18 | Body: 15
cm) ...")
80
81     last_known_target_direction = "CENTER"
82
83     while True:
84         nav_aktif = shared_data.get('nav_active', False) if shared_data
is not None else True
85
86         # 1. Baca Sensor Ultrasonik
87         jarak_mentah = ultrasonic.ukur_jarak_mentah()
88         if shared_data is not None:
89             if jarak_mentah is not None: shared_data['raw_distance'] =
round(jarak_mentah, 2)

```

```

90         current_factor = shared_data.get('faktor_koreksi',
ultrasonic.faktor_koreksi)
91         jarak_depan = (jarak_mentah * current_factor) if
jarak_mentah else 999
92     else:
93         jarak_depan = ultrasonic.get_jarak()
94
95     if not nav_aktif:
96         stop_robot()
97         time.sleep(0.1)
98         continue
99
100     # --- STATE 1: JARAK DEKAT (MUNDUR & HINDAR) ---
101     if jarak_depan <= 15:
102         stop_robot()
103         time.sleep(0.1)
104
105     # Cek apakah ini target finish (Harus berwarna target DAN
area besar)
106     vision_data = ambil_data_visi()
107     target_area = 0
108     if vision_data:
109         for obj in vision_data.get('objects', []):
110             if obj['label'] == 'TARGET':
111                 area = obj['bbox']['w'] * obj['bbox']['h']
112                 if area > target_area: target_area = area
113
114     if target_area > TARGET_REACHED_AREA and jarak_depan <= 12:
115         print(f"[FINISH] Target Terpenuhi (Area: {target_area})
| Stop.")
116         stop_robot()
117         if shared_data is not None: shared_data['nav_active'] =
False
118         continue
119     else:
120         # Jika jarak dekat tapi bukan target penuh -> Itu
Obstacle
121         print(f">> OBSTACLE TERDETEKSI ({jarak_depan:.1f} cm) |
Manuver Mundur...")
122         jalankan_motor(-V_SLOW, -V_SLOW)
123         time.sleep(0.5)
124         stop_robot()
125         time.sleep(0.2)
126
127         # Belok sedikit untuk mencari jalan lain

```

```

128         if last_known_target_direction == "LEFT":
129             print(">> MANUEVER: Belok KANAN menghindari
rintangan")
130             jalankan_motor(120, -100)
131         else:
132             print(">> MANUEVER: Belok KIRI menghindari rintangan
")
133             jalankan_motor(-100, 110)
134             time.sleep(0.3)
135             stop_robot()
136             time.sleep(0.2)
137             continue
138
139     # --- STATE 2: AMBIL GAMBAR & TENTUKAN ARAH ---
140     stop_robot() # Diam saat ambil gambar agar akurat
141     time.sleep(0.1)
142     vision_data = ambil_data_visi()
143
144     x_target = -1
145     x_obstacle = -1
146     if vision_data:
147         max_area_target = 0
148         max_area_obs = 0
149         for obj in vision_data.get('objects', []):
150             area = obj['bbox']['w'] * obj['bbox']['h']
151             if obj['label'] == 'TARGET' and area > MIN_AREA_TARGET:
152                 if area > max_area_target:
153                     max_area_target = area
154                     x_target = obj['bbox']['x'] + (obj['bbox']['w']
/ 2)
155             elif obj['label'] == 'OBSTACLE' and area >
MIN_AREA_OBSTACLE:
156                 if area > max_area_obs:
157                     max_area_obs = area
158                     x_obstacle = obj['bbox']['x'] + (obj['bbox']['w']
' ] / 2)
159
160     # --- STATE 3: EKSEKUSI GERAKAN ---
161     # PRIORITAS 1: Jika ada target, kejar target!
162     if x_target != -1:
163         error = x_target - 160
164         steer = hitung_PID(error)
165
166         v_kiri = V_BASE + steer
167         v_kanan = V_BASE - steer

```

```

168
169         # Logging lebih sensitif (ambang batas diturunkan dari 40
ke 20)
170         if error < -20:
171             last_known_target_direction = "LEFT"
172             print(f">> BELOK KIRI: Target X:{x_target:.1f} | L:{
v_kiri:.1f} R:{v_kanan:.1f}")
173         elif error > 20:
174             last_known_target_direction = "RIGHT"
175             print(f">> BELOK KANAN: Target X:{x_target:.1f} | L:{
v_kiri:.1f} R:{v_kanan:.1f}")
176         else:
177             last_known_target_direction = "CENTER"
178             print(f">> MAJU: Target X:{x_target:.1f} | L:{v_kiri:.1
f} R:{v_kanan:.1f}")
179
180         jalankan_motor(v_kiri, v_kanan)
181         time.sleep(0.25)
182
183         # PRIORITAS 2: Jika tidak ada target tapi ada rintangan,
hindari!
184         elif x_obstacle != -1:
185             # Safe margin dipersempit agar tidak terlalu penakut (75px)
186             if x_obstacle < (160 + 75) and x_obstacle > (160 - 75):
187                 if x_obstacle < 160:
188                     print(f">> HINDAR KANAN: Ada rintangan di kiri")
189                     jalankan_motor(V_BASE + 35, V_BASE - 35)
190                 else:
191                     print(f">> HINDAR KIRI: Ada rintangan di kanan")
192                     jalankan_motor(V_BASE - 25, V_BASE + 25)
193             else:
194                 print(f">> MAJU: Rintangan di luar jalur (Aman)")
195                 jalankan_motor(V_BASE, V_BASE)
196                 time.sleep(0.25)
197
198         else:
199             # PRIORITAS 3: Cari target berdasarkan memori
200             if last_known_target_direction == "LEFT":
201                 print(f">> SEARCH: Putar KIRI mencari target...")
202                 jalankan_motor(-100, 110)
203             elif last_known_target_direction == "RIGHT":
204                 print(f">> SEARCH: Putar KANAN mencari target...")
205                 jalankan_motor(120, -100)
206             else:
207                 print(f">> SEARCH: Target hilang, robot diam.")

```

```

208         stop_robot()
209         time.sleep(0.2)
210
211         stop_robot()
212         time.sleep(0.1) # Jeda stabilisasi
213
214 if __name__ == "__main__":
215     try:
216         main_loop()
217     except KeyboardInterrupt:
218         stop_robot()
219     finally:
220         motors.cleanup()

```

Listing 3: Kode Program Modul Navigasi Utama (navigator.py)

4 Pengujian dan Hasil Pembahasan

Pengujian dilakukan untuk mengukur keandalan sistem robotika yang telah dirancang. Pengujian mencakup kalibrasi akurasi sensor jarak serta pengujian navigasi pada lintasan uji coba.

4.1 Pengujian Akurasi Sensor Jarak

Pengujian sensor dilakukan dengan membandingkan nilai jarak pembacaan sensor ultrasonik dengan jarak pengukuran manual menggunakan penggaris. Hasil pengujian ditunjukkan pada Tabel 1.

Tabel 1: Data Pengujian Akurasi Sensor Jarak Ultrasonik HC-SR04.

No.	Jarak Aktual (cm)	Jarak Sensor (cm)	Selisih / Error (cm)	Persentase Error (%)
1	10.0	10.1	0.1	1.00
2	20.0	19.8	0.2	1.00
3	30.0	30.3	0.3	1.00
4	40.0	39.5	0.5	1.25
5	50.0	50.2	0.2	0.40
Rata-rata Error (%)				0.93%

Berdasarkan Tabel 1, terlihat bahwa sensor ultrasonik memiliki rata-rata error sebesar 0.93%, yang mana masih berada di bawah ambang batas toleransi sistem kendali (5%), sehingga sensor dinyatakan layak digunakan untuk sistem navigasi robot.

4.2 Analisis Pengujian Navigasi

Setelah dilakukan kalibrasi sensor, pengujian dilanjutkan dengan mengamati respon robot dalam mendeteksi penghalang di depannya. Robot berhasil mendeteksi dinding penghalang pada jarak ≈ 20 cm dan melakukan manuver belok kanan secara otomatis untuk menghindari tabrakan. Eksperimen dilakukan sebanyak 10 kali lintasan dengan persentase keberhasilan navigasi sebesar 90%.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem robotika yang telah dilakukan, dapat diambil beberapa kesimpulan sebagai berikut:

1. Prototipe robot berhasil dirancang dengan mengintegrasikan sensor ultrasonik HC-SR04, driver motor L298N, dan mikrokontroler Arduino Uno.
2. Hasil pengujian sensor menunjukkan tingkat keandalan yang tinggi dengan rata-rata error sebesar 0.93%.
3. Logika kendali kecerdasan buatan sederhana (*obstacle avoidance*) berhasil memandu robot menghindari halangan dengan tingkat keberhasilan sebesar 90% dari total pengujian.

5.2 Saran

Beberapa saran yang dapat diajukan untuk pengembangan sistem robot ini lebih lanjut adalah:

1. Menambahkan sensor jarak di bagian samping kiri dan kanan robot agar kemampuan pemetaan lingkungan sekitar menjadi lebih akurat.
2. Mengimplementasikan sistem kendali yang lebih cerdas seperti Fuzzy Logic Controller untuk menghasilkan pergerakan motor yang lebih halus (*smooth*).
3. Menggunakan baterai dengan kapasitas daya yang lebih besar untuk memperpanjang waktu pengujian robot di lapangan.

Daftar Pustaka

- [1] Adi Pratama and Budi Wijaya. “Desain dan Implementasi Robot Otonom untuk Navigasi Cerdas di Lingkungan Dinamis”. In: *Jurnal Teknologi Robotika Indonesia* 12.2 (2026), pp. 45–53. DOI: [10.12345/jttri.v12i2.6789](https://doi.org/10.12345/jttri.v12i2.6789).